

OOP

Events and Delegate

Collections, Generic, Generic Collection

Exception Handling

GC

File IO, Serialization

7 { Reflection  
(Sharp) Feature → LINK → 10  
ADO.NET  
mvz

panhalemugdha@gmail.com

8275006278

# Reflection :-

Reading type metadata of assembly dynamically. and we can also invoke this assembly. type / functionality.

## Assembly Structure

C# .dll

|                                                                                                       |
|-------------------------------------------------------------------------------------------------------|
| <del>PE Header</del>                                                                                  |
| Assembly MD.                                                                                          |
| Resource MD.                                                                                          |
| Type <br>Metadata = |
| MSIL                                                                                                  |

.dll ~~exe~~

```

class Cmath
{
    void Add (x, int y)
    void Sub (int x, int y)
}
Cmath. del

```

"D:\MyApp\ Cmath.dll"

```

Assembly asm = Assembly.LoadFrom ("path");
Type[] alltypes = asm.GetTypes();

```

```

Type type = alltypes[0];
(- UUDemo. Cmath)

```

```

type. IsClass = true; ✓
type. IsPublic = true; ✓
type. IsSealed = false;
type. IsInherited = false;
type. IsInternal = false;

```

```

type. IsStatic = false;
type. IsAbstract = false;
type. IsInterface = false;
type. FullName → "Demo.Cmath"
type. Name = "Cmath"
type. IsGeneric = false;

```

Property Info [] all props = type.GetProperties();

Constructor Info [] all ctors = type.GetConstructors();

Field Info [] all fields

Methods Info [] all methods = type.<sup>(meth. —)</sup>GetMethods();

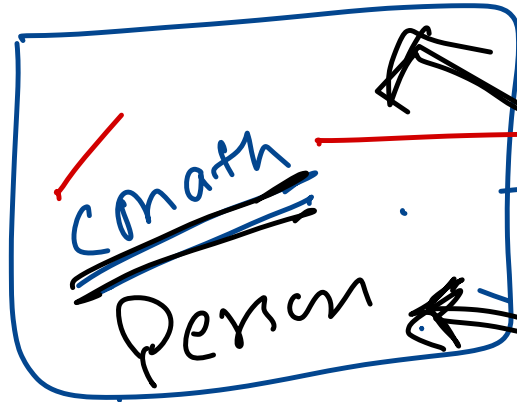
Method Info method = allmethods [0]; <sup>Add</sup> <sub>Sub.</sub>  
↓

Parameter Info

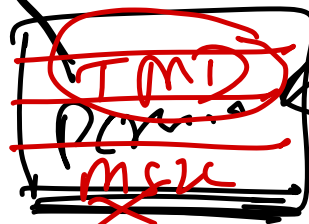
[ ] all params = method.<sup>method.Name</sup>GetParameters(); <sub>(Add.)</sub>  
[2, 4]

# Reflection

Path: D:\CMath.dll



Load into memory



Assembly class.



ch

OODemo.exe

```
void Add (System.Int32 x, System.Int32 y)
```

```
void Sub (System.Int32 x, System.Int32 y)
```

